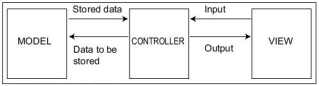

Figure 1: Model View Controller

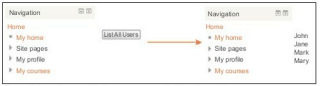

Figure 2: Interface to display usernames of all registered users

process. In schools and universities, it can be used by the faculty members to post study material, interact with students, prepare tests, etc. Students may use it to collaborate with each other to work on projects, upload assignments, take tests, etc. The server-side code for Moodle is written using PHP.

The problem

Let's assume that Moodle is hosted on an Apache server. Create a plug-in for Moodle to allow the current user to view all the users registered on the site. The users' information is stored in a MySQL database on the server.

The solution

Design: On the click of a button, the names of all the users registered on the site should be displayed on the screen. Using the MVC pattern, we can divide the design into three components:

1. Forms to display the button and the result.
2. Class/es that provide/s functions to store and retrieve the data to be used for processing the input and producing the output.
3. A controller to maintain a link between the forms and the classes. The controller will decide which function to call and what data to send or retrieve to/ from the class/es.

The model is independent of the view, i.e., it is unaware of the current form being displayed to the user. The model interacts only with the controller, although it is, in a way, associated with the view as well.

Please note, since Moodle provides its own APIs to connect to the database and perform database operations, we need not explicitly write the code to do so.

Code: The user should be provided with an interface to view all the users. This can be achieved with the help of a form that consists of a button with the text 'List All Users'.

When the user clicks on the button, the controller will retrieve all the users' details from the model, which will retrieve the details from the database on the server. The result will be given back to the view to make appropriate changes on the form.

Hence, the model class will contain methods as follows (the entire code has not been specified):

```
<?php
require_once(''); //files required to use the Moodle API
class Model
{
  public function get_users()
      {
      global $DB; //used to access the data manipulation APIs
       $queryString = "SELECT * FROM table_users";

 //table_users contains the details of all the users
  $result = $DB->get_records_sql($queryString);
                          return $result;
            }

           //other functions
}
```

The following is the code for the controller:

```
<?php
            //display the form
require_once(''); //files required to use the Moodle API
            require_once('users_form.class.php');

            $usersForm = new Users_Form();

 //display the standard headings on the page, such as the
page title, heading etc.
$usersForm->display_headings();

//check if button has been clicked. It will return null if
button has not been clicked
if($usersForm->get_data() == null)
                        $usersForm->display();
            else {
                        //retrieve users
                        $modelObject = new Model;
                        $users  = $modelObject->get_users();
                        $userForm->display_users($users);
            }
?>
```

The following is the code for the view, with a list and a button:

```
<?php
            //include the files required to use the Moodle API
            require_once('');

            //create a Moodle form
```